

Detecção de Placas Veiculares com MobileNetV1 e Detector por Grade Embarcado em ESP32-S3

Carlos Vinícius dos Santos Mesquita, Alisson Jaime Sales Barros,
Victor Guedes Alves Teixeira e Kaique Ferreira Braga Silva
TI0147 — Fundamentos de Processamento Digital de Imagens
Professor: Paulo Cesar Cortez
Universidade Federal do Ceará

Resumo—Este trabalho apresenta um sistema completo de detecção de placas veiculares baseado na arquitetura MobileNetV1 com uma cabeça de detector por grade (saída espacial $7 \times 7 \times 5$), treinado em TensorFlow/Keras e embarcado em uma placa ESP32-S3 com câmera OV5640. O modelo foi quantizado em INT8 e convertido para TensorFlow Lite, viabilizando a inferência diretamente no microcontrolador. Avaliamos o sistema em três modalidades (imagem, vídeo e captura em tempo real com a ESP32-CAM), comparamos o desempenho entre PC e dispositivo embarcado, e aplicamos pós-processamento com filtragem por confiança e segmentação dos caracteres. Os resultados, obtidos sobre cinco execuções, mostram IoU médio de $0,731 \pm 0,005$ no conjunto combinado e *recall* de 0,922, com a quantização INT8 preservando a qualidade (IoU praticamente idêntico ao modelo Keras). A inferência custa $\sim 4\text{--}7$ ms no PC contra ~ 117 s na ESP32-S3, evidenciando o gargalo do hardware embarcado. O acréscimo de imagens negativas reduziu os falsos positivos a zero no limiar 0,95.

Index Terms—detecção de objetos, MobileNetV1, TensorFlow Lite, sistemas embarcados, ESP32, quantização INT8, processamento digital de imagens.

I. INTRODUÇÃO

A detecção de objetos é uma tarefa central da Visão Computacional, consistindo em localizar objetos de interesse por meio de *bounding boxes*. Sua execução em sistemas embarcados, contudo, é desafiadora: dispositivos como a ESP32 possuem forte restrição de memória e processamento. Arquiteturas da família MobileNet, baseadas em convoluções separáveis em profundidade (*depthwise separable convolutions*), foram projetadas para esse cenário, reduzindo parâmetros e custo computacional.

Este trabalho desenvolve, treina e embarca um detector de placas veiculares usando MobileNetV1 com uma cabeça de detecção por grade, e o avalia nas três modalidades exigidas (imagem, vídeo e tempo real). Além do embarcamento na ESP32-S3, o mesmo modelo é executado em PC para comparação de desempenho, e o sistema é complementado por pós-processamento (filtragem por confiança e segmentação dos caracteres da placa).

II. BASE DE DADOS

Foram utilizadas duas bases de detecção de objetos de classe única (*plate*), com anotações no formato YOLO (centro, largura e altura normalizados): **Base 1**, fornecida pelo professor, e **Base 2**, a base pública *Brazil Plates Detector*

Tabela I
DIVISÃO DA BASE UTILIZADA (UMA CLASSE: *plate*).

Conjunto	Treino	Validação
Base 1 (professor)	350	60
Base 2 (Brazil Plates)	200	12
Mix ponderado	972	72
Negativas (sem placa)	12	4

(Roboflow, licença CC BY 4.0), escolhida pela aderência ao problema. As bases foram combinadas com ponderação da Base 2 para reforçar exemplos difíceis. A Tabela I resume a divisão efetivamente utilizada. Adicionalmente, incorporamos **16 imagens negativas** (cenhas sem placa) para reduzir falsos positivos.

III. ARQUITETURA E DETECTOR POR GRADE

A arquitetura é a **MobileNetV1** com fator de largura (*width multiplier*) $\alpha = 0,5$ e entrada $224 \times 224 \times 3$. Sobre o *backbone* pré-treinado adicionamos uma **cabeça de detecção por grade** que produz uma saída espacial $7 \times 7 \times 5$: a imagem é dividida em uma grade 7×7 e cada célula prevê uma confiança de objetos (*objectness*) e os quatro parâmetros da caixa (c_x, c_y, w, h). A detecção final corresponde à célula de **maior confiança**, e a caixa global é reconstruída por $c_x = (\text{col} + c_x^{\text{local}})/7$ e $c_y = (\text{lin} + c_y^{\text{local}})/7$. Essa formulação resolveu o problema de localização da região correta da placa observado em abordagens de regressão direta de uma única caixa. A escolha do MobileNetV1 $\alpha = 0,5$ equilibra precisão e tamanho, essencial para o embarcamento.

IV. METODOLOGIA

A. Pré-processamento

As imagens são redimensionadas para 224×224 e normalizadas para o intervalo $[-1, 1]$ por $x_{\text{norm}} = x/127,5 - 1$. Avaliamos dois modos de cor: **RGB** e **gray3** (escala de cinza replicada em três canais), este último por vezes mais robusto a variações de cor. Durante o treino aplicamos *Data Augmentation*: espelhamento horizontal, variações de brilho/contraste, ruído e desfoque.



Figura 1. Pós-processamento: ROI da placa, máscara de segmentação e caracteres segmentados (OpenCV).

B. Treinamento

O treino emprega o otimizador Adam em duas fases (*transfer learning* seguido de *fine-tuning* da cabeça): 12 e 50 épocas, *batch* 8, com *ReduceLROnPlateau* e *EarlyStopping*, e função de perda combinando confiança (*objectness*) e regressão da caixa. Cada configuração foi executada **cinco vezes**, reportando-se média e desvio padrão (Seção V). O ambiente de treino foi o Google Colab (TensorFlow 2.20.0, Python 3.12, GPU NVIDIA Tesla T4).

C. Pós-processamento

Aplicamos **filtragem por confiança**: a detecção só é considerada válida quando a confiança ultrapassa o limiar 0,95 e se mantém por **2 frames consecutivos**, reduzindo disparos espúrios. Em seguida, realizamos a **segmentação dos caracteres** da placa (obrigatória) por técnicas clássicas de OpenCV: recorte da ROI, equalização local (CLAHE), realce por *BlackHat*, limiarização de Otsu/adaptativa e filtragem de componentes conectados (Figura 1).

D. Conversão e Quantização para a ESP32

O modelo Keras foi convertido para TensorFlow Lite em três variantes (Float32, *dynamic* e INT8). A quantização **INT8 pós-treinamento** foi escolhida para o embarcamento. O arquivo `.tflite` (2,35 MB) foi convertido em um `array C++ (xnd -i)`, gerando um cabeçalho de **2.468.584 bytes**. No firmware (Arduino/ESP-IDF, biblioteca `TensorFlowLite_ESP32`), o *tensor arena* (~3,4 MB) é alocado na PSRAM e uma tabela de partições personalizada de 5 MB acomoda o aplicativo (3,74 MB de *flash*). A placa é uma **ESP32-S3** (16 MB *flash*, 8 MB PSRAM OPI, CPU a 240 MHz) com câmera **OV5640**.

E. Condições Experimentais

O **treinamento e a avaliação das métricas** foram realizados no Google Colab (TensorFlow 2.20.0, Python 3.12, GPU NVIDIA Tesla T4). Os **testes em PC** (execução LiteRT/TFLite) usaram um notebook com CPU Intel Core i7-14700HX, 16 GB de RAM, Windows 11 (64 bits), Python 3.14. O **dispositivo embarcado** é uma ESP32-S3 (16 MB *flash*, 8 MB PSRAM OPI, CPU a 240 MHz) com câmera OV5640, usando a biblioteca `TensorFlowLite_ESP32` (Arduino-ESP32 3.3.8 / ESP-IDF 5.5).

Tabela II
MÉTRICAS EM CINCO EXECUÇÕES (MÉDIA ± DESVIO).

Métrica	Base 1	Base 2	Mix
IoU médio	0,768 ± 0,002	0,543 ± 0,030	0,731 ± 0,005
Recall (IoU ≥ 0,5)	0,973 ± 0,013	0,667 ± 0,075	0,922 ± 0,014
F1@0,5	0,983	0,750	0,944

Score balanceado: 0,655 ± 0,015.

Tabela III
TEMPO DE INFERÊNCIA, TAMANHO E IOU (CONJUNTO MIX).

Plataforma / modelo	Tempo	Tamanho	IoU
Keras (PC, float)	~96,9 ms	n/d	0,736
TFLite Float32 (PC)	~6,5 ms	8,75 MB	0,736
TFLite INT8 (PC, LiteRT)	~4–7 ms	2,35 MB	0,732
TFLite INT8 (ESP32-S3)	~117 s	2,35 MB	n/d

Tabela IV
FALSOS POSITIVOS EM IMAGENS SEM PLACA POR LIMIAR.

Limiar	0,50	0,90	0,95	0,98
Falsos positivos	4/4	1/4	0/4	0/4

V. RESULTADOS

A. Desempenho de detecção (cinco execuções)

A Tabela II apresenta as métricas médias e desvios das cinco execuções. O modelo atinge *recall* elevado na Base 1 e no conjunto combinado; o desempenho menor na Base 2 reflete sua maior dificuldade e menor tamanho.

B. Comparação PC vs. ESP32 e validação da quantização

A Tabela III compara plataformas. A quantização INT8 **preservou a qualidade** (IoU praticamente idêntico ao Keras), reduzindo o modelo de 8,75 MB (Float32) para 2,35 MB. No PC a inferência custa poucos milissegundos; na ESP32-S3, ~117 s por *frame*, cerca de $1,7 \times 10^4$ vezes mais lenta. O gargalo decorre dos *kernels* de referência da biblioteca TFLite Micro e do acesso à PSRAM, não do modelo em si.

C. Efeito das imagens negativas

A Tabela IV mostra os falsos positivos sobre imagens sem placa em função do limiar. No limiar adotado (0,95) não há falsos positivos, e os exemplos positivos não foram degradados (confiança média ~ 0,99). Medições na própria ESP32-S3 confirmaram o ganho: cenas sem placa caíram de ~ 0,62 (modelo antigo) para 0,42–0,58, enquanto uma placa real bem enquadrada atinge 0,98–0,99.

D. Avaliação nas três modalidades

Imagem. A Figura 2 mostra a detecção em uma imagem: a caixa é posicionada corretamente sobre a placa e o mapa de confiança da grade destaca a célula correta.

Vídeo. O modelo foi executado quadro a quadro sobre uma sequência de vídeo (Figura 3), atingindo confiança de até 0,996 no melhor *frame*, com taxa de processamento de centenas de FPS no PC.



Figura 2. Detecção em imagem: entrada, caixa prevista (confiança $\approx 1,0$) e mapa de confiança da grade 7×7 .



Figura 3. Detecção em vídeo (melhor *frame*, confiança 0,996).

Tempo real. A captura ao vivo com a OV5640 acoplada à ESP32-S3 produziu detecção embarcada com confiança 0,97, disparando o envio automático da imagem anotada (Telegram). A Figura 4 ilustra a correta *rejeição* de um quadro sem placa nítida (confiança abaixo de 0,95).

VI. DISCUSSÃO

A quantização INT8 mostrou-se vantajosa: reduziu o modelo em $\sim 73\%$ sem perda mensurável de IoU, viabilizando o embarcamento. A principal limitação é o **tempo de inferência na ESP32-S3** (~ 117 s/frame), atribuível aos *kernels* de referência da TFLite Micro e à latência da PSRAM, e não ao tamanho do modelo, já que o mesmo arquivo roda em milissegundos no PC. As imagens negativas eliminaram os falsos positivos no limiar de operação. A qualidade da câmera OV5640 e o desfoque por movimento afetam a confiança, mitigados por bom enquadramento e pela exigência de detecção estável por múltiplos *frames*.



Figura 4. Rejeição correta: quadro sem placa nítida, confiança abaixo do limiar.

VII. CONCLUSÃO E TRABALHOS FUTUROS

Desenvolvemos um detector de placas com MobileNetV1 e cabeça por grade, embarcado e funcional na ESP32-S3, avaliado nas três modalidades e comparado ao PC. A solução é viável e precisa, com a quantização preservando a qualidade. Como trabalhos futuros: (i) acelerar a inferência embarcada com *kernels* otimizados (ESP-NN) ou entrada menor (96×96); (ii) alternativamente, transmitir a captura da ESP32-CAM para inferência em servidor (*frontend* fluido); (iii) ampliar a base de negativas com fotos capturadas pela própria ESP-CAM; e (iv) reconhecimento óptico (OCR) dos caracteres já segmentados.

REFERÊNCIAS

- [1] A. G. Howard *et al.*, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017.
- [2] M. Abadi *et al.*, "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Disponível: <https://www.tensorflow.org>
- [3] "TensorFlow Lite / LiteRT Guide." [Online]. Disponível: <https://www.tensorflow.org/lite>
- [4] Roboflow, "Brazil Plates Detector Dataset." [Online]. Disponível: <https://universe.roboflow.com/testes-bbb7m/brazil-plates-detector-zn2t0>
- [5] F. Chollet *et al.*, "Keras." [Online]. Disponível: <https://keras.io>
- [6] Espressif, "ESP-IDF / ESP32-S3 Technical Reference." [Online]. Disponível: <https://docs.espressif.com>